English Learning Platform

Documentacion Tecnica del Sistema

v1.0 | Django + React | 2025

Stack: Django REST Framework | React | PostgreSQL | OpenAI Whisper | RapidFuzz

1. Vision General del Sistema

Esta plataforma es un sistema de aprendizaje de ingles adaptativo con cuatro habilidades evaluables: Reading, Speaking, Listening Shadowing y Listening Comprehension. El sistema ajusta automaticamente la dificultad de los ejercicios segun el desempeno del alumno usando un motor de ventana deslizante (sliding window).

1.1 Niveles del curso

El curso se organiza en cuatro niveles CEFR. Cada nivel contiene una secuencia fija de ejercicios que el alumno debe completar en orden, sin posibilidad de saltarse pasos.

Nivel	CEFR	Descripcion	XP requerido para avanzar
A1	Beginner	Vocabulario y estructuras basicas	Acumular suficiente XP con ejercicios medios/altos
A2	Elementary	Expresion simple y comprension	Definido por el equipo al armar el curso
B1	Intermediate	Comunicacion en situaciones cotidianas	Mayor proporcion de ejercicios dificiles
B2	Upper-Int.	Fluidez y comprension de textos complejos	Incluye simulacro TOEFL

1.2 Subdificultad dentro de cada nivel

Cada nivel se subdivide en tres categorias de dificultad. Esta subdivision determina cuantos puntos XP otorga cada ejercicio:

Dificultad	XP maximo	Ejemplo score 78/100	XP obtenido
Facil	10 pts	78 / 100 = 0.78	10 x 0.78 = 7.8 pts
Medio	20 pts	78 / 100 = 0.78	20 x 0.78 = 15.6 pts
Dificil	30 pts	78 / 100 = 0.78	30 x 0.78 = 23.4 pts

Formula XP

xp_earned = max_pts_dificultad * (score / 100)

Ejemplo: ejercicio DIFICIL, score 78 => 30 * 0.78 = 23.4 pts (se redondea a 23)

2. Motor Adaptativo (Sliding Window)

El sistema evalua continuamente el desempeno del alumno usando una ventana de los ultimos N ejercicios para decidir si subir, mantener o bajar la dificultad. Este mismo motor aplica para TODAS las habilidades: Reading, Speaking, Listening Shadowing y Listening Comprehension.

2.1 Reglas de adaptacion

Promedio ventana	Condicion	Accion del sistema	Ventana minima
avg >= 16	Buen desempeno	Subir a dificultad ALTA	3 ejercicios
avg 10 - 15.9	Desempeno normal	Mantener dificultad actual	3 ejercicios
avg < 10	Bajo desempeno	Bajar a dificultad BAJA	3 ejercicios

Ejemplo de sesion real: alumno en dificultad MEDIA con scores 18, 17, 19, 19, 18. Promedio = 18.2. Como avg >= 16 por mas de 3 ejercicios, el sistema comienza a mostrar ejercicios de dificultad ALTA.

2.2 Implementacion en Django (logica backend)

```
# adaptive_engine.py

WINDOW_SIZE = 3
THRESHOLD_UP   = 16.0   # sube dificultad
THRESHOLD_DOWN = 10.0   # baja dificultad

DIFFICULTY_MAX_XP = {
    'EASY':   10,
    'MEDIUM': 20,
    'HARD':   30,
}

def calculate_xp(score: float, difficulty: str) -> int:
    max_xp = DIFFICULTY_MAX_XP[difficulty]
    return round(max_xp * (score / 100))

def get_next_difficulty(user_id: int, skill: str) -> str:
    # Obtiene los ultimos WINDOW_SIZE intentos del usuario para esta habilidad
    recent = Attempt.objects.filter(
        user_id=user_id, skill=skill
    ).order_by('-created_at')[:WINDOW_SIZE]

    if len(recent) < WINDOW_SIZE:
        return 'MEDIUM'  # default hasta tener suficientes datos

    avg = sum(a.xp_earned for a in recent) / len(recent)
```

```
    if avg >= THRESHOLD_UP:
        return 'HARD'
    elif avg < THRESHOLD_DOWN:
        return 'EASY'
    else:
        return recent[0].difficulty  # mantener actual
```

3. Habilidades Evaluables

Todas las habilidades comparten el mismo motor XP y el mismo motor adaptativo. Lo que cambia es como se obtiene el score 0-100.

3.1 Reading

Como funciona

- Se presenta un texto al alumno.
- Se muestran preguntas de opcion multiple sobre el texto.
- Score = (respuestas correctas / total preguntas) * 100
- No hay texto libre para evitar problemas de validacion.

Calculo de score

```
# reading_service.py

def calculate_reading_score(answers: dict, question_set: list) -> float:
    correct = sum(
        1 for q in question_set
        if answers.get(str(q.id)) == q.correct_answer
    )
    return round((correct / len(question_set)) * 100, 2)
```

3.2 Speaking

Como funciona

- Se muestra un texto o frase al alumno.
- El alumno graba su voz pronunciando la frase.
- React envia el audio a Django.
- Django llama a OpenAI Whisper para transcribir el audio.
- Se compara la transcripcion con el texto esperado usando RapidFuzz.
- Score = fuzz.ratio(expected, transcribed) => ya devuelve 0-100.

Flujo tecnico

```
# speaking_service.py
import openai
from rapidfuzz import fuzz

def transcribe_and_score(audio_file, expected_text: str) -> dict:
    client = openai.OpenAI(api_key=settings.OPENAI_API_KEY)
```

```
    transcription = client.audio.transcriptions.create(
        model='whisper-1',
        file=audio_file,
        language='en'
    )

    transcribed = transcription.text.strip().lower()
    expected    = expected_text.strip().lower()
    score       = fuzz.ratio(expected, transcribed)  # 0-100

    return {
        'transcribed_text': transcription.text,
        'score': score,
        'api_raw': transcription.model_dump()
    }
```

3.3 Listening — Shadowing (Opcion B)

Como funciona

- Audio CORTO: 5-10 segundos, una sola frase.
- Repeticiones del audio: ILIMITADAS (el alumno escucha todas las veces que necesite).
- El alumno graba su voz repitiendo lo que escucho.
- Se procesa exactamente igual que Speaking (Whisper + RapidFuzz).
- Score = misma formula que Speaking.

Diferencia clave con Speaking

En Speaking, el alumno VE el texto y lo pronuncia.
En Listening Shadowing, el alumno ESCUCHA el audio y lo repite SIN ver el texto.
Esto entrena la comprension auditiva y la pronunciacion simultaneamente.

3.4 Listening — Comprehension (Opcion A)

Como funciona

- Audio MAS LARGO: 30-60 segundos, un parrafo completo.
- Repeticiones del audio: MAXIMO 3 (contador visible para el alumno).
- Se presentan preguntas de opcion multiple sobre el contenido del audio.
- Score = mismo calculo que Reading.

Campo adicional en la BD

```
-- En la tabla questions:
max_replays INT DEFAULT NULL
```

```
  -- NULL  = ilimitado  (Shadowing)
  -- 3     = maximo 3   (Comprehension)

-- Tipos de question expandidos:
type VARCHAR(25) NOT NULL CHECK (type IN (
    'SPEAKING',
    'READING',
    'LISTENING_SHADOWING',
    'LISTENING_COMPREHENSION'
))
```

Control del contador en React

```
// ListeningPlayer.jsx
const [replaysLeft, setReplaysLeft] = useState(question.max_replays);

const handlePlay = () => {
    if (replaysLeft === null) {
        audioRef.current.play();  // ilimitado
        return;
    }
    if (replaysLeft > 0) {
        audioRef.current.play();
        setReplaysLeft(prev => prev - 1);
    }
};

// En el template:
// replaysLeft === null ? 'ilimitado' : `${replaysLeft} reproducciones restantes`
```

4. API de Audio — OpenAI Whisper

Whisper es el modelo de transcripcion de voz de OpenAI. Recibe un archivo de audio y devuelve el texto transcrito. Es la pieza central para Speaking y Listening Shadowing.

4.1 Costo estimado

Escenario	Usuarios	Ejercicios/dia	Audio prom.	Costo/mes aprox.
Demo universitaria	10	5 c/u	10 seg	~$1.50 USD
Proyecto pequeno	50	10 c/u	15 seg	~$11 USD
En crecimiento	200	10 c/u	15 seg	~$45 USD

Para el proyecto universitario

OpenAI otorga $5 USD de credito gratis al registrarse.
Con ese credito pueden cubrir todo el desarrollo, pruebas y la demo final sin gastar un centavo.

4.2 Grabacion de audio en React

```
// useAudioRecorder.js (custom hook)
import { useState, useRef } from 'react';

export const useAudioRecorder = () => {
    const [isRecording, setIsRecording] = useState(false);
    const [audioBlob, setAudioBlob]     = useState(null);
    const mediaRecorderRef = useRef(null);
    const chunksRef        = useRef([]);

    const startRecording = async () => {
        const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
        const recorder = new MediaRecorder(stream);
        chunksRef.current = [];

        recorder.ondataavailable = (e) => chunksRef.current.push(e.data);
        recorder.onstop = () => {
            const blob = new Blob(chunksRef.current, { type: 'audio/webm' });
            setAudioBlob(blob);
        };

        recorder.start();
        mediaRecorderRef.current = recorder;
        setIsRecording(true);
    };

    const stopRecording = () => {
```

```
        mediaRecorderRef.current?.stop();
        setIsRecording(false);
    };

    return { isRecording, audioBlob, startRecording, stopRecording };
};
```

4.3 Endpoint Django para Speaking / Shadowing

```
# views/speaking_view.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from .speaking_service import transcribe_and_score
from .adaptive_engine import calculate_xp, get_next_difficulty
from ..models import SpeakingAttempt, Question, UserProgress

class SpeakingAttemptView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, question_id):
        question = Question.objects.get(id=question_id)
        audio    = request.FILES.get('audio')

        result = transcribe_and_score(audio, question.correct_answer)
        score  = result['score']
        xp     = calculate_xp(score, question.difficulty)

        # Guardar intento
        SpeakingAttempt.objects.create(
            user           = request.user,
            question       = question,
            expected_text  = question.correct_answer,
            transcribed_text = result['transcribed_text'],
            transcription_match = score / 100,
            api_response_raw = result['api_raw'],
            score            = score,
            xp_earned        = xp,
        )

        # Actualizar XP global
        progress = UserProgress.objects.get(user=request.user)
        progress.total_xp = max(0, progress.total_xp + xp)
        progress.save()

        # Calcular siguiente dificultad
        next_diff = get_next_difficulty(request.user.id, 'SPEAKING')

        return Response({
            'score':           score,
            'xp_earned':       xp,
            'transcribed':     result['transcribed_text'],
```

```
            'next_difficulty': next_diff,
        })
```

5. Flujo Completo de una Sesion

Este es el ciclo de vida de un ejercicio desde que el alumno entra hasta que el sistema actualiza su progreso.

5.1 Flujo general

1. El alumno inicia sesion (JWT auth).
2. El frontend consulta al backend: siguiente ejercicio segun nivel y dificultad adaptativa.
3. El backend llama a get_next_difficulty() y devuelve una pregunta del tipo y dificultad correspondiente.
4. El alumno completa el ejercicio (segun el tipo: graba voz, selecciona opcion, escucha audio).
5. El frontend envia la respuesta al backend.
6. El backend calcula el score (0-100).
7. El backend calcula xp_earned = max_xp_dificultad * (score / 100).
8. Se guarda el intento en la BD (speaking_attempts / reading_attempts).
9. Se actualiza user_progress.total_xp.
10. Se reevalua la ventana deslizante para decidir la siguiente dificultad.
11. El backend responde con: score, xp_earned, feedback, next_difficulty.
12. El frontend muestra retroalimentacion al alumno.

5.2 Retroalimentacion al alumno

Cada ejercicio completado debe mostrar retroalimentacion inmediata con:

- Score obtenido (ej: 78/100).
- XP ganado en este ejercicio.
- XP total acumulado.
- Para Reading: cuales respuestas fueron correctas e incorrectas y por que.
- Para Speaking/Shadowing: el texto transcrito vs el texto esperado.
- Indicador de siguiente dificultad si hubo cambio.

5.3 Historial de ejercicios

El alumno puede revisar todos sus intentos anteriores (lectura solamente, no repetir). El historial muestra:

- El ejercicio original.
- Su respuesta o transcripcion.
- El score obtenido.
- La fecha y hora del intento.

Sin embargo, el alumno SI puede volver a intentar cualquier ejercicio para practicar (farmear XP), aunque ya tenga 10/10. Los reintentos tambien suman XP y cuentan para la ventana adaptativa.

6. Sistema XP y Desbloqueo de Examenes

6.1 Reglas de XP

Evento	XP
Ejercicio FACIL completado	max 10 pts * (score/100)
Ejercicio MEDIO completado	max 20 pts * (score/100)
Ejercicio DIFICIL completado	max 30 pts * (score/100)
Racha activa (streak >= 3 dias)	+15 XP bonus por ejercicio
Primera vez en una categoria	+50 XP sorpresa
Intento extra fallido en Speaking	-3 XP adicional por intento
Minimo absoluto de XP	0 XP (nunca baja de cero)

6.2 Desbloqueo de examenes

Cuando el total_xp del alumno alcanza el umbral definido en la tabla exams, el sistema automaticamente:

13. Agrega el exam_id al array user_progress.unlocked_exam_ids.
14. Inserta una notificacion de tipo EXAM_UNLOCKED.
15. El desbloqueo es permanente: aunque el XP baje, el examen sigue desbloqueado.

```
# exam_unlock_service.py

def check_and_unlock_exams(user_id: int):
    progress = UserProgress.objects.get(user_id=user_id)
    unlocked = set(progress.unlocked_exam_ids or [])

    pending_exams = Exam.objects.filter(
        xp_required__lte=progress.total_xp
    ).exclude(id__in=unlocked)

    for exam in pending_exams:
        unlocked.add(exam.id)
        Notification.objects.create(
            user_id = user_id,
            type    = 'EXAM_UNLOCKED',
            message = f'Has desbloqueado el examen: {exam.level} - {exam.type}'
        )

    progress.unlocked_exam_ids = list(unlocked)
    progress.save()
```

7. Generacion de Audios para Listening

Los audios para los ejercicios de Listening se generan UNA SOLA VEZ con Text-to-Speech y se almacenan estaticamente. Esto elimina costos recurrentes.

7.1 Opciones recomendadas

Opcion	Costo	Calidad	Recomendado para
Google TTS	Gratis hasta 1M chars/mes	Buena	Generacion masiva inicial
OpenAI TTS	~$0.015 / 1k chars	Muy natural	Audios de alta calidad
ElevenLabs	Gratis limitado	Excelente	Demo / presentacion

Estrategia recomendada para el proyecto universitario

1. Usar Google TTS para generar todos los audios del curso una sola vez.
2. Guardar los archivos en el servidor o en un bucket S3/Storage gratuito.
3. Guardar la URL del audio en el campo content_url de la tabla lessons.

Costo total: $0.

8. Arquitectura del Sistema

8.1 Stack tecnologico

Capa	Tecnologia	Responsabilidad
Frontend	React + Axios	UI, grabacion de audio, reproductor
Backend	Django REST Framework	Logica de negocio, XP, adaptacion
Base de datos	PostgreSQL	Persistencia de datos
Auth	JWT (djangorestframework-simplejwt)	Autenticacion y sesiones
Transcripcion	OpenAI Whisper API	Speaking y Listening Shadowing
Scoring	RapidFuzz (Python)	Comparacion texto esperado vs transcrito

8.2 Endpoints principales

Metodo	Endpoint	Descripcion	Respuesta clave
POST	/api/auth/login/	Login JWT	access_token, refresh_token
GET	/api/exercises/next/	Siguiente ejercicio adaptativo	question, difficulty, type
POST	/api/speaking/attempt/	Enviar audio Speaking	score, xp_earned, transcribed, next_difficulty
POST	/api/reading/attempt/	Enviar respuestas Reading	score, xp_earned, correct_answers
POST	/api/listening/shadowing/	Enviar audio Shadowing	score, xp_earned, transcribed
POST	/api/listening/comprehension/	Enviar respuestas Comprehension	score, xp_earned, correct_answers
GET	/api/progress/	Progreso del alumno	total_xp, streak, nivel, unlocked_exams

8.3 Instalacion de dependencias

```
# Backend
pip install django djangorestframework djangorestframework-simplejwt
pip install openai rapidfuzz psycopg2-binary python-decouple

# Frontend
```

```
npm install axios react-router-dom

# Variables de entorno (.env)
OPENAI_API_KEY=sk-...
DATABASE_URL=postgresql://user:pass@localhost:5432/english_platform
SECRET_KEY=django-secret-key-aqui
DEBUG=True
```

9. Decisiones Pendientes del Equipo

Estos puntos requieren una decision antes de comenzar a codificar:

#	Decision pendiente	Opciones
1	Umbral de XP para subir de nivel (A1→A2, etc.)	Definir cuantos XP se necesitan por nivel
2	Tamano de ventana del motor adaptativo	Por defecto: 3 ejercicios. Puede ajustarse a 5.
3	Proveedor de TTS para generar audios	Google TTS (gratis) vs OpenAI TTS (pago)
4	Almacenamiento de archivos de audio	Servidor local vs S3/GCS bucket
5	Quien crea el contenido del curso (preguntas, textos)	El equipo o usar un generador automatico con IA

Recomendacion final

Para la demo universitaria: Google TTS + almacenamiento local + credito gratuito de OpenAI.
Esto mantiene el costo total del proyecto en $0 durante el desarrollo.
El sistema esta disenado para escalar a produccion sin cambiar la logica de negocio.